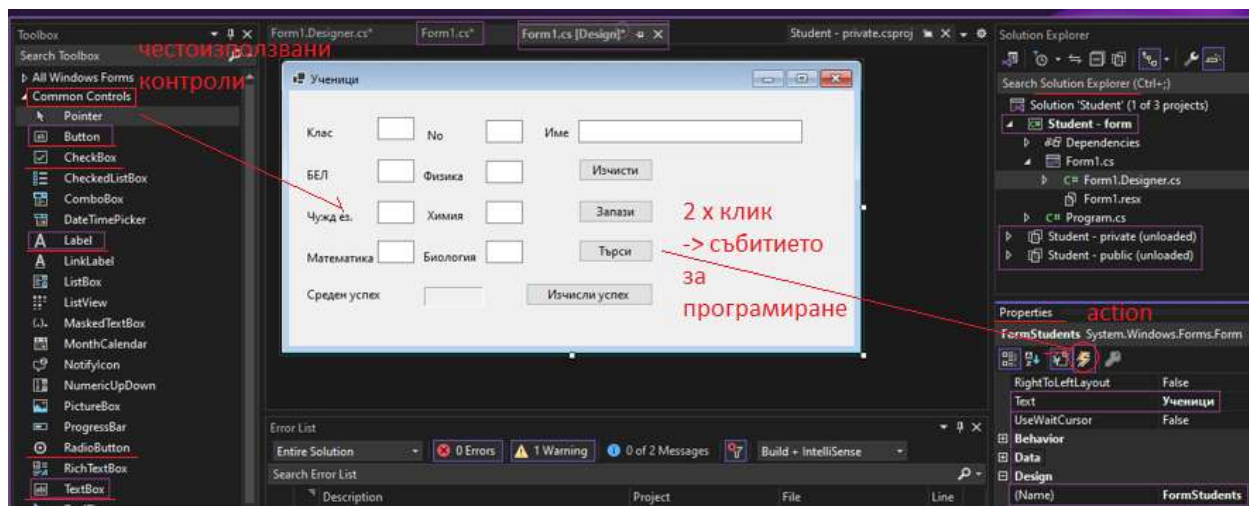


ЧИСЛОВИ ТИПОВЕ ДАННИ В C#

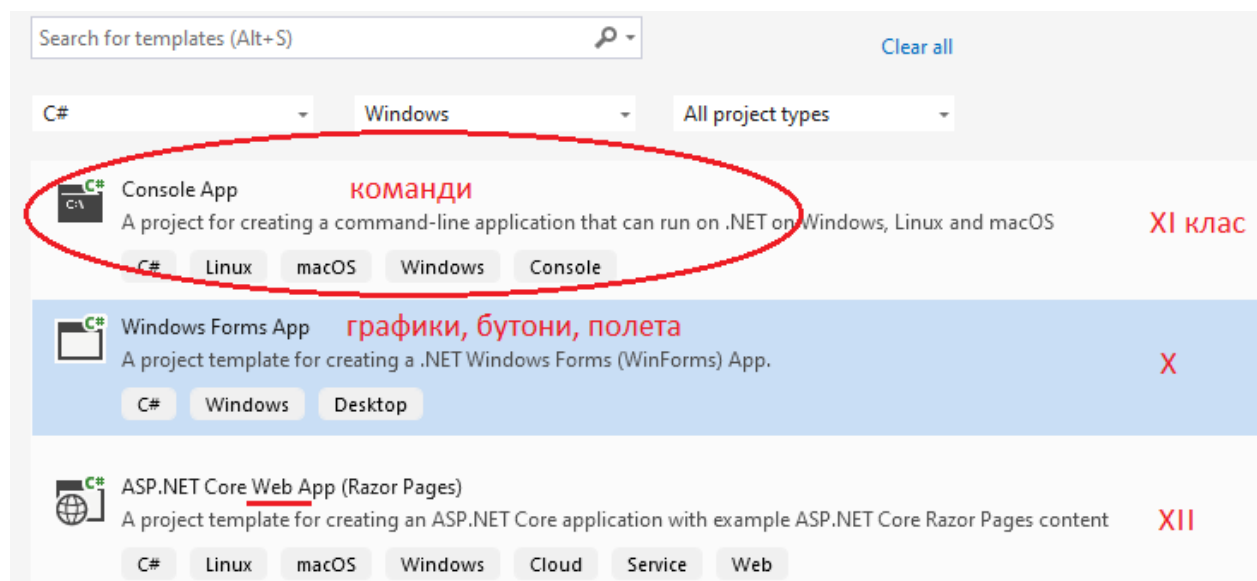
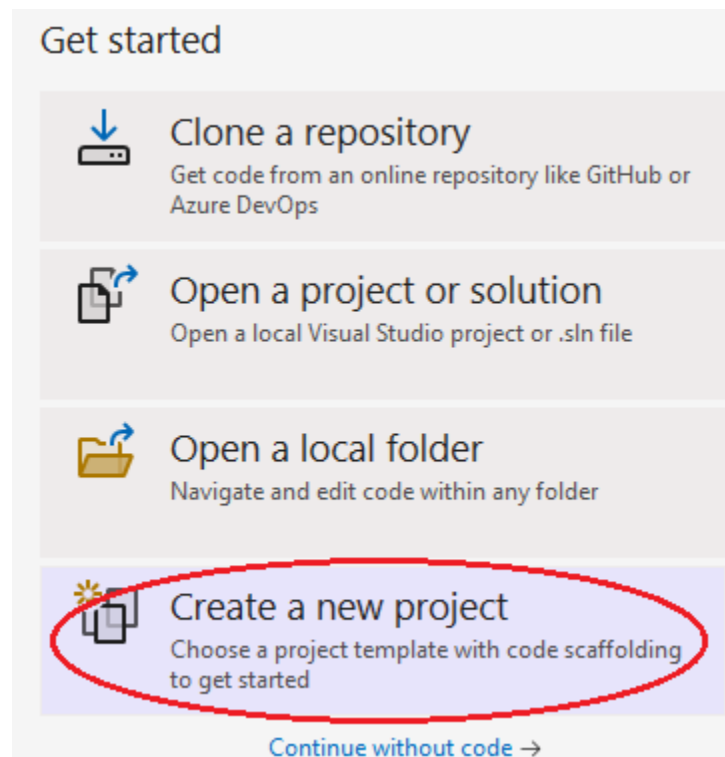
1. Въведение във Visual Studio

А) форми – Windows Forms – силно развит графичен интерфейс GUI

Провличаме D&D (drag and drop) от левия панел контролите (елементите), които искаме, подреждаме ги във формата. Задаваме желаните свойства от десния панел Properties – цвят, размер... именуваме всяка от тях правилно, така че да можем да я намерим по време на писане на кода. Желателно е да запазим типа, а просто да заместим цифрата, например Button01 -> ButtonClose, TextBox01 -> TextBoxName и т.н. Всеки един клик върху елемент отваря действие (action) или още събитие, което може да се програмира на C# (какво да се случи при активация).



В) конзола / терминал Console App – команден интерфейс



При именуването на проекта няма конкретна конвенция, която се спазва. Важното е името да е описателно. Не е желателно да започва с цифри – само за учебни цели.

Configure your new project

Console App C# Linux macOS Windows Console

Project name
 име на урока, PascalCase

Location
 вашата папка в D:

Solution name ⓘ

☒ Place solution and project in the same directory **.sln - файла**

Project will be created in "D:\Exercises\11 class\01_DataTypes\" **в същата, а не по-горна директория (извън папката)**

Имената трябва да са описателни

админът е забранил промени в C:

Конвенции за именуване:

PascalCase – начална главна буква - методи (от името на математика Блез Паскал)

camelCase – начална малка буква – променливи (като гърбиците на камила)

snake_case – БД, т.к. интервалите са проблем (долни черти като змия)

kebab-case – в интернет URL

Additional information

Console App C# Linux macOS Windows Console

Framework ⓘ

☐ Do not use top-level statements ⓘ

☐ Enable native AOT publish ⓘ

.NET 6.0 – отпада файл Startup.cs

.NET 8.0 – скрива се Main {} – метода

С всяка по-нова версия, независимо дали е като настройки на самата ОС или език за програмиране, нещата се опростяват/скриват все повече.

```

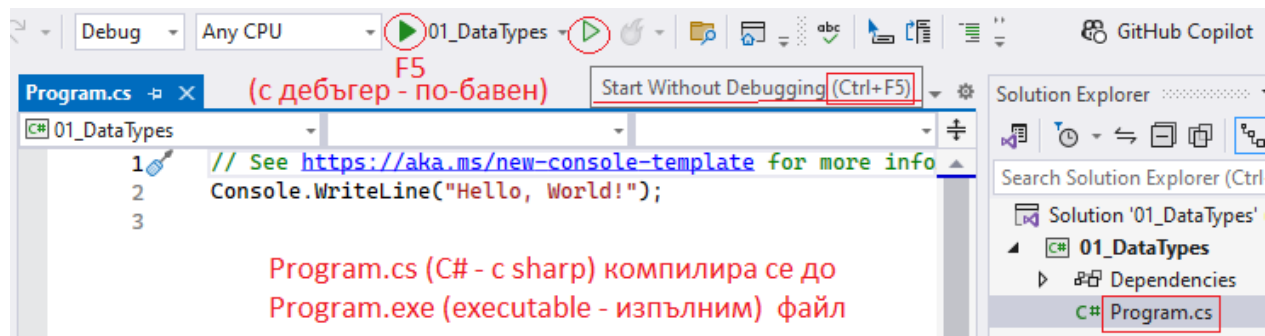
public class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        var app = builder.Build();

        app.MapGet("/", () => "Hello World!");

        app.Run();
    }
}

```

Програмата е последователност от команди. Нейният текст се нарича „сурс код“ (source – извор). Програмата с разширение .cs се компилира до изпълним .exe. Може да бъде стартирана с един от двата зелени триъгълни бутона, като Ctrl + F5 стартира по-бързо, защото е без дебъгер, докато самостоятелно F5 го включва (и забавя).



Тип име = стойност;

```

// type, name, value
int i = 3;

```

Присвояването се случва отдясно наляво!

Типовете могат да бъдат прости (вградени) и сложни (класове/обекти, потребителски дефинирани от съставянето на няколко други прости, интерфейси и т.н.).

Понеже компютърът работи само с 0 и 1, това оформя двоична бройна система и всичко се представя като степени на двойката.

С малка буква **b** = бит **bit** (обикновено за скорост на интернет)

С главна буква **B** = Байт **Byte** (обикновено за количество памет)

$2^0 = 1$
$2^1 = 2$
$2^2 = 4$
$2^3 = 8$
$2^4 = 16$
$2^5 = 32$
$2^6 = 64$
$2^7 = 128$
$2^8 = 256$
$2^9 = 512$

$1024 = 2^{10}$ - т.е. КИЛО в случая не е точно 1000 както доста хора се бъркат.

единица	съкращение (англ.)	значение
бит	bit (b)	0 или 1
байт	Byte (B)	8 бит
килобайт	KByte (KB)	1024 байта
мегабайт	MByte (MB)	1024 килобайта
гигабайт	GByte (GB)	1024 мегабайта

кило-грам = 1000 грама, кило-метър = 1000 метра

Име	Символ	Количество
килобайт	kB	10^3
мегабайт	MB	10^6
гигабайт	GB	10^9
терабайт	TB	10^{12}

2. Прости / вградени типове данни

А) Целочислени типове

byte b = 255; // 0 до 255 (1 b байт) вместо от 1 до 256 в обратна посока, за да включим 0 <-
sbyte sb = -128; // -128 до 127 (1 байт) - делим 256 на 2, за да добавим знаци + - s = signed знаков)
short s = -32768; // -32,768 до 32,767 (2 байта)
ushort us = 65535; // 0 до 65,535 (2 байта) – всички unsigned (беззнакови са двойно по-големи)
int i = 2147483647; // -2.1e9 до 2.1e9 (4 байта) – цели числа (- 0 +)
uint ui = 4294967295; // 0 до 4.3e9 (4 байта)
long l = 9223372036854775807; // -9.2e18 до 9.2e18 (8 байта)
ulong ul = 18446744073709551615; // 0 до 1.8e19 (8 байта)

Б) Дробни числа (с плаваща запетая/точка)

float f = 3.14F; // ~6-9 цифри (4 байта) – винаги буквата f накрая
double d = 3.1415926535; // ~15-17 цифри (8 байта) – най-често използван, но има уязвимост
decimal dec = 123.456M; // 28-29 значими цифри (16 байта) – най-точен и голям, за финанси

Ако по време на работа решим да сменим типа на данните и преминем от по-малък към по-голям, това може да стане автоматично (имплицитно) без проблеми, например

```
int a = 10;  
long b = a;
```

Ако, обаче, искаме да преминем от по-голям тип към по-малък, това става явно с уточняване (експлицитно) към кой точно тип да смалим, като обикновено има загуба на стойности:

```
double c = 2.4;  
int e = (int)c; // уточняваме какъв да бъде новия тип и ще има срязване на числото до c = 2.
```

в) Други

```
char c = 'A'; // Unicode символ (2 байта) -  
bool flag = true; // true /false (1 байт) или съответно 1/0  
string s = "Hello world!" // текст
```

var – variable (променлива) – когато не сме сигурни какъв тип да използваме, просто използваме

него и той се намира автоматично по ориентир според присвоената стойност, например

```
var name = "John"; // string
var age = 25;      // int
```

const – ключова дума, чрез която уточняваме, че ще използваме константа. Прието е константите да са с начална главна буква, за да се различават (PascalCase), но не всички букви да са главни. Те не могат да се променят и ако се опитаме да им присвоим нова стойност по-надолу в кода, ще хвърли грешка (червено дъгообразно подчертаване).

```
const int K = 5;
```

```
K = K + 2;
```

Най-често използваме:

- **int** - цяло число: 1, 2, 3, 4, 5, ...
- **double** - дробно число: 0.5, 2.17, -1.5, ...
- **string** - текст (низ): "Въведете число", "Hi", "OK", ... - с двойни кавички
- **char** - символ: 'A', '#', '@', '+', ... - с единични кавички

Всичко, което въвеждаме в конзолата и което се разпечатва от конзолата, е ТЕКСТ (string), затова, когато работим с числа, те трябва да бъдат преобразувани / парснати с parse.

Аритметични операции:

+ - събиране на числа или конкатениране (слепване) на низове/текст/string

- изваждане

* умножение

/ - целочислено деление (остатъкът не се връща) например $9 / 2 = 4$ и остатък 1, който се игнорира

% - деление с остатък, например $9 \% 2 = 1$

!= означава различно

== дали е равно?

Демонстрационни задачи:

```
// type, name, value

int a = 3;
int b = 4;

Console.WriteLine(a + b);

int s = a + b;

Console.WriteLine(s);

double d = 0.1;
double e = 0.2;

Console.WriteLine(d + e); // уязвимост в паметта при double

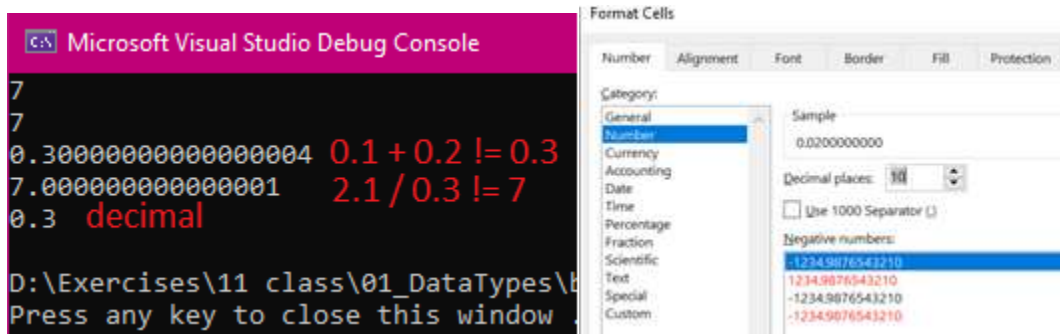
double f = 2.1;
double g = 0.3;

Console.WriteLine(f/g); // още един пример - особено около 3

decimal d1 = 0.1M;
decimal e1 = 0.2M;

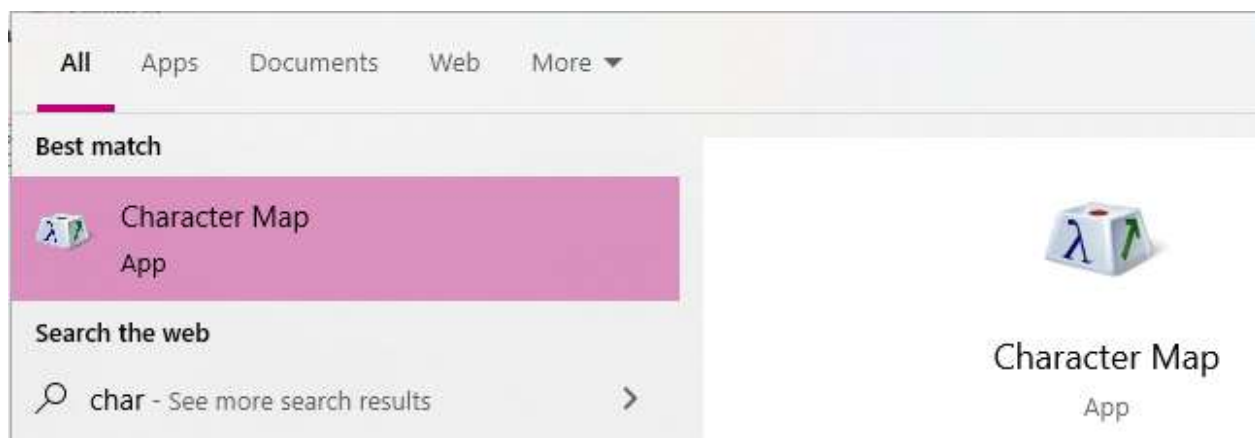
Console.WriteLine(d1 + e1); // при финанси Decimal!!!
```

Резултат / изход:

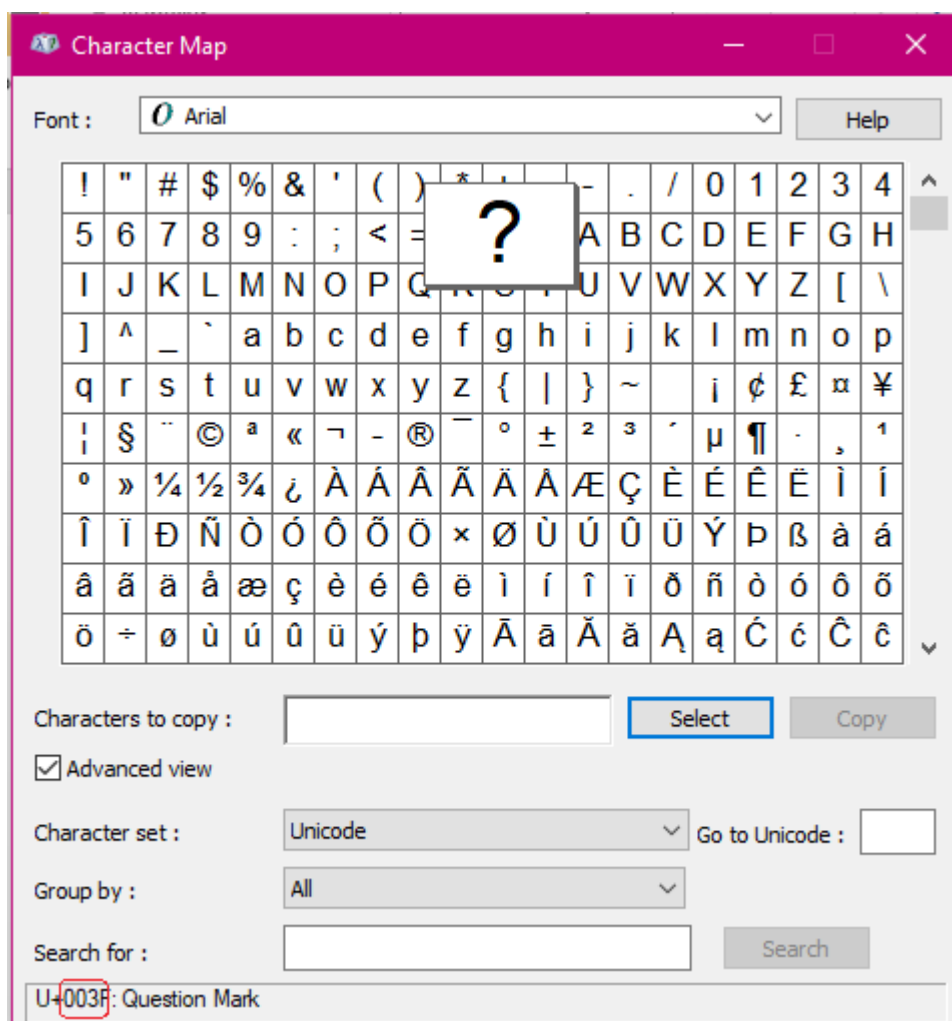


Вижда се, че тип decimal е по-точен от double. Той използва повече памет и изчислява по-бавно, но при финансови операции се използва само и единствено той, точно поради уязвимости в паметта. Той е единственият тип с плаваща запетая, който е заложен в програмата Excel, върху която се крепи огромна част от финансовата система.

cw – snippet за автодописване на Console.WriteLine();



char – съкратено от character – имаме право на 1 символ от огромната таблица на Unicode, която съдържа няколко хиляди символа и много езици.



Всеки символ има уникален код, който може да бъде отпечатан и с него, т.е. вместо знака ? може да използваме неговия униккод 003F, като кажем на компилатора да го пропусне и разчете чрез \u

```
char f1 = '?';
Console.WriteLine(f1);
char f2 = '\u003F'; // \u is Unicode for ?
Console.WriteLine(f2);
```

Резултатът е един и същи.

При създаването си, компютрите са работели само с ASCII таблица, която съдържа големи и малки латински букви, числа и някои специални символи – общо 128 знака.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

С разпространението на компютрите, тя се оказва крайно недостатъчна за останалите езици – кирилица, японски и т.н. Затова се създава Unicode, която към момента съдържа 154 998 символа.

```
char c1 = '@';
char c2 = '\u263A'; // :) emoticon
```

```
Console.WriteLine(c1 + " " + c2);
Console.WriteLine(c1 + c2); // sums unicode
```

Результати



```
string cat = Console.ReadLine();  
  
Console.WriteLine("My cat's name is "  
    + cat);  
  
int age = int.Parse(Console.ReadLine());  
  
Console.WriteLine("Her age is " + age);
```